Studio Nx

S1-11a 재생 오디오 그래프 — 함수 설명 합본 (JS 288~296)

2026-10-19

목차

1. ai-notes/javascript/S1-11a - 재생 오디오 그래프/288_audio_mixer.md
2. ai-notes/javascript/S1-11a - 재생 오디오 그래프/289_audio_playback.md
3. ai-notes/javascript/S1-11a - 재생 오디오 그래프/290_audio_session.md
4. ai-notes/javascript/S1-11a - 재생 오디오 그래프/291_audio_synth.md
5. ai-notes/javascript/S1-11a - 재생 오디오 그래프/292_audio_notes_instruments.md
6. ai-notes/javascript/S1-11a - 재생 오디오 그래프/293_audio_effects.md
7. ai-notes/javascript/S1-11a - 재생 오디오 그래프/294_audio_effect_rack.md
8. ai-notes/javascript/S1-11a - 재생 오디오 그래프/295_audio_meter.md
9. ai-notes/javascript/S1-11a - 재생 오디오 그래프/296_sequencer_audio.md

JavaScript 288 — 트랙 믹서 (`audio/mixer.js`)

항목	내용
파일	`studionx/frontend/static/js/audio/mixer.js`
작업	S1-11a 재생 오디오 그래프: 트랙 볼륨, 팬, 뮤트, 솔로, 마스터, 이펙트 체인 자리 (FR-251, FR-252)
작성	Claude (AI), 2026-10-19, 사용자 검토
이전, 다음	이전 287, 다음 289 클립 재생

1. 하는 일

트랙마다 "소스, 이펙트 체인, 볼륨, 팬" 순서로 노드를 이어 두고, 모든 트랙을 마스터로 모아 최종 출력으로 보냅니다. 뮤트와 솔로 규칙, dB 변환은 소리를 쓰지 않는 순수 함수로 따로 두었습니다. 컨텍스트를 인자로 받기 때문에 실시간 재생과 WAV 출력(S1-11b)이 같은 코드를 씁니다.

```
악기, 클립 ─▶ [source] ─▶ 이펙트 체인 ─▶ [input: 볼륨, 뮤트, 솔로] ─▶ [pan] ┐
                                                                              ├─▶ [master] ─▶ 마스터 이펙
트 ─▶ [out] ─▶ 스피커
(다른 트랙도 같은 모양) ──────────────────────────────────────────────────────┘
```

2. 언제 쓰나

쓰는 곳	하는 일
`PlaybackSession` (290)	재생을 시작할 때 믹서를 만들고, 재생 중에는 150ms마다 `applyTracks`를 불러 볼륨과 뮤트를 다시 반영합니다.
`ClipPlayer`, `InstrumentRack`	`trackInput(trackId)`으로 트랙의 입구 노드를 받아 소리를 보냅니다.
`EffectRack` (294)	`setTrackEffects`, `setMasterEffects`로 이펙트 체인을 바꿔 끼웁니다.
레벨 미터 (295)	`meterFor(trackId)`로 볼륨과 팬을 거친 소리의 크기를 읽습니다.

3. 코드

3.1 dbToGain

```
export function dbToGain(db) {
  const v = Number(db);
  if (!isFinite(v) || v <= -60) return 0;
  return Math.pow(10, v / 20);
}
```

3.2 audibleTrackIds

```
export function audibleTrackIds(tracks) {
  const list = (tracks || []).filter(function (t) { return t.track_type !== 'MASTER'; });
```

```
  const anySolo = list.some(function (t) { return !!t.is_solo; });
  return list.filter(function (t) {
    if (t.is_muted) return false;
    return anySolo ? !!t.is_solo : true;
  }).map(function (t) { return t.track_id; });
}
```

3.3 createStereoGain

```
export function createStereoGain(context) {
  const g = context.createGain();
  g.channelCount = 2;
  g.channelCountMode = 'explicit';
  g.channelInterpretation = 'speakers';
  return g;
}
```

3.4 MixerGraph 의 노드 만들기

```
constructor(context, destination) {
  this.context = context;
  this.master = context.createGain();          // 마스터 볼륨
  this.out = context.createGain();             // 마지막 출력 (마스터 이펙트 뒤) — 소리를 재거나 녹음할 때 여기에 붙임
  this.out.connect(destination != null ? destination : context.destination);
  this.master.connect(this.out);
  this.masterChain = [];                       // 마스터 이펙트 (master → 체인 → out)
  this.strips = {};   // trackId -> { source, input(볼륨), pan, chain }
  this.taps = {};     // 트랙 키 → LevelTap (미터)
}
```

```
strip(trackId) {
  const key = String(trackId);
  if (this.strips[key]) return this.strips[key];
  const ctx = this.context;
  const source = createStereoGain(ctx);   // 악기 · 클립이 모이는 곳 (이펙트 체인의 입구, 모노도 스테레오로 늘림)
  const input = ctx.createGain();      // 트랙 볼륨 · 뮤트 · 솔로 (이펙트 체인 뒤)
  const pan = typeof ctx.createStereoPanner === 'function' ? ctx.createStereoPanner() : null;
  source.connect(input);
  if (pan) { input.connect(pan); pan.connect(this.master); }
  else input.connect(this.master);
  this.strips[key] = { source: source, input: input, pan: pan, chain: [] };
  return this.strips[key];
}
```

```
trackInput(trackId) {
  return this.strip(trackId).source;
}
```

3.5 이펙트 체인 바꿔 끼우기

```
_rewire(head, tail, oldList, newList) {
  try { head.disconnect(); } catch (e) { /* 연결이 없었음 */ }
  oldList.forEach(function (fx) { try { fx.output.disconnect(); } catch (e) { /* 무시 */ } });
  let prev = head;
  newList.forEach(function (fx) { prev.connect(fx.input); prev = fx.output; });
  prev.connect(tail);
}
```

```
setTrackEffects(trackId, list) {
  const s = this.strip(trackId);
  this._rewire(s.source, s.input, s.chain, list);
  s.chain = list.slice();
}
```

```
setMasterEffects(list) {
  this._rewire(this.master, this.out, this.masterChain, list);
  this.masterChain = list.slice();
}
```

3.6 트랙 값 반영과 미터

```
applyTracks(tracks) {
  const audible = {};
  audibleTrackIds(tracks).forEach(function (id) { audible[String(id)] = true; });
  const self = this;
  const now = this.context.currentTime;
  (tracks || []).forEach(function (t) {
    if (t.track_type === 'MASTER') {
      self.master.gain.setTargetAtTime(dbToGain(t.volume_db), now, 0.01);
      return;
    }
    const s = self.strip(t.track_id);
    const gain = audible[String(t.track_id)] ? dbToGain(t.volume_db) : 0;
    s.input.gain.setTargetAtTime(gain, now, 0.01);   // 뚝 끊기는 소리(클릭)를 막으려고 아주 짧게 이어서
바꿈
    if (s.pan) s.pan.pan.setTargetAtTime(Math.max(-1, Math.min(1, Number(t.pan) || 0)), now, 0.01);
  });
}
```

```
meterFor(trackId) {
  const key = String(trackId);
  if (this.taps[key]) return this.taps[key];
  let node;
  if (key === 'master') node = this.out;
  else { const s = this.strip(key); node = s.pan || s.input; }
  this.taps[key] = new LevelTap(this.context, node);
  return this.taps[key];
}
```

4. 최신 문법으로 쓰면

실제 코드는 전통 문법으로 썼고, 최신 문법으로 쓰면 아래와 같습니다 (참고용입니다).

```
// audibleTrackIds 를 화살표 함수와 구조 분해로 쓴 모양
export const audibleTrackIds = (tracks = []) => {
  const list = tracks.filter(({ track_type }) => track_type !== 'MASTER');
  const anySolo = list.some(({ is_solo }) => !!is_solo);
  return list
    .filter(({ is_muted, is_solo }) => !is_muted && (!anySolo || !!is_solo))
    .map(({ track_id }) => track_id);
};
```

전통 문법 (실제 코드)	최신 문법
`function (t) { return ...; }`	`(t) => ...`
`tracks || []`	기본 매개변수 `tracks = []`
`t.track_type`	구조 분해 `{ track_type }`
`destination != null ? destination : context.destination`	`destination ?? context.destination`

5. 왜 이렇게

선택	이유
뮤트와 솔로 규칙을 순수 함수로 뺐습니다	소리 없이 Node에서 시험할 수 있고, 규칙이 바뀌어도 한곳만 고치면 됩니다.
트랙마다 `source`와 `input`을 따로 두었습니다	이펙트는 볼륨(페이더)보다 앞에 있어야 합니다. 그래서 `source`가 체인의 입구이고 `input`이 볼륨입니다.
마스터에 `master`와 `out`을 따로 두었습니다	마스터 이펙트(리미터 등)가 마스터 볼륨 뒤에 들어갑니다. 소리를 재거나 녹음할 때는 모든 처리가 끝난 `out`에 붙입니다.
입구를 스테레오로 고정했습니다 (`createStereoGain`)	모노 소리가 `StereoPannerNode`를 지나면 가운데에서 -3dB가 됩니다. 미리 스테레오로 늘려 두면 가운데에서는 크기가 그대로입니다. 자세한 내용은 문서 83을 봅니다.
값을 `setTargetAtTime`으로 바꿉니다	볼륨이 갑자기 바뀌면 "틱" 하는 잡음이 납니다. 10ms의 시간 상수로 부드럽게 잇습니다.
연결을 바꿀 때 `disconnect` 후 다시 잇습니다	노드 연결은 추가만 되고 덮어쓰기가 없어서, 옛 연결을 끊어야 소리가 두 번 나지 않습니다.
볼륨 -60dB 이하를 완전한 무음으로 둡니다	DB 범위의 아래 끝이 -60dB이고, 이 값을 "끔"으로 쓰기 때문입니다.

6. 시험

- `docs/handoff/s1-11a_tests/mixer_playback_unit.mjs`는 `dbToGain`과 `audibleTrackIds`를 확인합니다. 0dB는 1이고, -6dB는 약 0.5이며, -60dB는 0입니다. 솔로가 켜지면 솔로 트랙만 들리고, 뮤트가 솔로보다 우선하며, MASTER는 규칙에서 빠집니다.
- `docs/handoff/s1-11a_tests/ui.mjs`는 실제 화면에서 뮤트하면 무음이 되고, 해제하면 소리가 돌아오며, 다른 트랙이 솔로이면 이 트랙이 무음이 되는지 확인합니다.
- `docs/handoff/s1-11a_tests/ui_fx.mjs`는 마스터 트랙에 붙인 Limiter가 최종 출력의 피크를 줄이는지 확인합니다.

JavaScript 289 — 오디오 클립 재생 (`audio/playback.js`)

항목	내용
파일	`studionx/frontend/static/js/audio/playback.js`
작업	S1-11a 재생 오디오 그래프: 타임라인의 오디오 클립을 실제 소리로 재생 (FR-205)
작성	Claude (AI), 2026-10-19, 사용자 검토
이전, 다음	이전 288 믹서, 다음 290 재생 세션

1. 하는 일

오디오 클립을 "언제, 샘플의 어디부터, 얼마나" 울릴지 계산하고, 그 계산대로 `AudioBufferSourceNode`를 예약합니다. 계산은 순수 함수이고, 예약은 `ClipPlayer`가 합니다.

```
클립 { start_tick, length_tick, offset_tick, gain_db, is_looped, pitch_semitones }
        │
        ├─ clipsStartingIn  이번 단계에서 새로 시작하는 클립을 고릅니다
        ├─ clipsCovering    재생 위치가 한가운데 걸친 클립을 고릅니다 (중간부터 재생)
        ▼
planClipPlayback  when(울릴 시각), offsetSec(샘플 안 시작 위치), durationSec(길이)
        ▼
ClipPlayer.schedule  샘플 버퍼 → 게인(클립 gain) → 트랙 입구
```

2. 언제 쓰나

쓰는 곳	하는 일
`PlaybackSession.start`	재생을 시작할 때 시작 위치에 걸친 클립을 `clipsCovering`으로 골라 중간부터 울립니다.
`PlaybackSession._onStep`	스케줄러가 한 단계(1/16박)를 넘길 때마다 `clipsStartingIn`으로 새로 시작할 클립을 골라 예약합니다.
루프 되감기	루프 끝에서 되감기면 `stopAll(time)`으로 옛 소리를 끊고, 되감긴 위치에 걸친 클립을 다시 시작합니다.

3. 코드

3.1 클립 고르기

```
export function clipsStartingIn(clips, fromTick, toTick) {
  return (clips || []).filter(function (c) {
    return c.start_tick >= fromTick && c.start_tick < toTick;
  });
}
```

```
export function clipsCovering(clips, tick) {
  return (clips || []).filter(function (c) {
    return c.start_tick < tick && c.start_tick + c.length_tick > tick;
```

```
  });
}
```

3.2 planClipPlayback

```
export function planClipPlayback(clip, nowTick, nowTime, baseBpm, changes) {
  const startTick = Math.max(clip.start_tick, nowTick);
  const skippedTicks = startTick - clip.start_tick;
  const toSec = function (a, b) {
    return ticksToSecondsWithChanges(b, baseBpm, changes) - ticksToSecondsWithChanges(a, baseBpm, change
s);
  };
  const when = nowTime + toSec(nowTick, startTick);
  const offsetSec = toSec(0, (clip.offset_tick || 0) + skippedTicks);
  const durationSec = toSec(startTick, clip.start_tick + clip.length_tick);
  return { when: when, offsetSec: offsetSec, durationSec: durationSec };
}
```

3.3 ClipPlayer

```
schedule(clip, nowTick, nowTime, baseBpm, changes) {
  if (clip.sample_id == null) return false;
  const buffer = this.getBuffer(clip.sample_id);
  if (!buffer) return false;
  const plan = planClipPlayback(clip, nowTick, nowTime, baseBpm, changes);
  if (!(plan.durationSec > 0)) return false;
  // 샘플보다 뒤에서 시작하면 소리가 없음 (루프가 아닐 때)
  if (!clip.is_looped && plan.offsetSec >= buffer.duration) return false;

  const ctx = this.context;
  const src = ctx.createBufferSource();
  src.buffer = buffer;
  src.loop = !!clip.is_looped;
  const semis = Number(clip.pitch_semitones) || 0;
  if (semis !== 0) src.playbackRate.value = Math.pow(2, semis / 12);
  const gain = ctx.createGain();
  gain.gain.value = dbToGain(clip.gain_db || 0);
  src.connect(gain);
  gain.connect(this.trackInput(clip.track_id));

  const when = Math.max(plan.when, ctx.currentTime);
  src.start(when, plan.offsetSec % (buffer.duration || 1));
  src.stop(when + plan.durationSec);
  const self = this;
  src.onended = function () {
    const i = self.active.indexOf(src);
    if (i >= 0) self.active.splice(i, 1);
  };
  this.active.push(src);
  return true;
}
```

```
stopAll(atTime) {
  const when = atTime != null ? atTime : 0;
```

```
  this.active.slice().forEach(function (src) {
    try { src.stop(when); } catch (e) { /* 이미 끝난 소스 */ }
    if (atTime == null) { try { src.disconnect(); } catch (e) { /* 무시 */ } }
  });
  if (atTime == null) this.active = [];
}
```

4. 최신 문법으로 쓰면

```
// clipsStartingIn 을 화살표 함수와 기본 매개변수로 쓴 모양
export const clipsStartingIn = (clips = [], fromTick, toTick) =>
  clips.filter(({ start_tick }) => start_tick >= fromTick && start_tick < toTick);

// planClipPlayback 의 오프셋 계산을 nullish 병합으로 쓴 모양
const offsetSec = toSec(0, (clip.offset_tick ?? 0) + skippedTicks);
```

전통 문법 (실제 코드)	최신 문법
`(clip.offset_tick || 0)`	`clip.offset_tick ?? 0`
`atTime != null ? atTime : 0`	`atTime ?? 0`
`function (c) { return ...; }`	`(c) => ...`

5. 왜 이렇게

선택	이유
계산을 순수 함수로 분리했습니다	시각과 위치 계산은 틀리기 쉬워서 소리 없이 숫자로 시험합니다. 중간부터 재생, 템포 변경 같은 경우도 같은 함수로 처리합니다.
시작 시각을 `max(when, currentTime)`으로 잡습니다	이미 지난 시각으로 예약하면 소리가 빠지거나 한꺼번에 울립니다. 지금으로 당겨서 울립니다.
재생 중간에 걸친 클립은 샘플을 건너뛰고 시작합니다	재생선이 클립 한가운데일 때 처음부터 울리면 위치가 어긋납니다. `offsetSec`에 이미 흐른 길이를 더합니다.
샘플이 없으면 조용히 건너뜁니다	샘플 내려받기에 실패해도 재생선과 다른 소리는 계속되게 하려는 것입니다.
정지는 `stop` 뒤 `disconnect`입니다	소스 노드는 한 번 쓰면 다시 쓸 수 없어서, 끊고 목록을 비웁니다.
루프 되감기는 시각을 지정해 끊습니다	미리 예약해 둔 소리 중 되감기 이전 구간만 끊어야 하므로, `stopAll(time)`에 되감기 시각을 줍니다.
음정은 `playbackRate`로 바꿉니다	반음 올림은 `2^(반음/12)`배 빠르게 재생하는 것과 같습니다. 길이도 같이 짧아집니다 (워프는 지원하지 않음).

6. 시험

- `docs/handoff/s1-11a_tests/mixer_playback_unit.mjs`는 120BPM에서 960틱이 0.5초인 기준으로 세 경우를 확인합니다. 정상 시작이면 `when`이 그대로이고 오프셋이 0입니다. 중간부터 재생이면 오프셋에 건너뛴 길이와 클립 `offset_tick`이 더해집니다. 아직 먼 클립이면 `when`이 그만큼 뒤입니다.
- `docs/handoff/s1-11a_tests/ui.mjs`는 440Hz 사인파 클립을 재생해 마스터 피크가 0.2에서 0.6 사이인지, 정지하면 무음인지 확인합니다.

JavaScript 290 — 재생 세션 (`audio/session.js`)

항목	내용
파일	`studionx/frontend/static/js/audio/session.js`
작업	S1-11a 재생 오디오 그래프: 엔진, 믹서, 클립, 노트, 이펙트를 묶어 재생, 일시정지, 정지를 소리로 바꿈 (FR-201, FR-205)
작성	Claude (AI), 2026-10-19, 사용자 검토
이전, 다음	이전 289 클립 재생, 다음 291 Synth

1. 하는 일

`PlaybackSession`은 소리 쪽 부품을 모두 모아 "재생 버튼 하나"로 동작하게 하는 지휘자입니다. 페이지의 값(트랙, 클립, BPM, 루프)은 `deps`로 주입받기 때문에 화면 없이도 시험할 수 있습니다.

```
PlaybackSession
  ├ AudioEngine      시계와 스케줄러(lookahead), 메트로놈
  ├ MixerGraph       트랙 믹서와 마스터 (288)
  ├ ClipPlayer       오디오 클립 재생 (289)
  ├ InstrumentRack   MIDI 노트를 악기로 연주 (292)
  ├ NotePlayer       노트 예약 (292)
  └ EffectRack       이펙트 체인 (294)

start(fromTick) ─▶ 샘플을 모두 내려받아 디코딩 ─▶ 믹서와 이펙트를 맞춤 ─▶ 엔진 시작
스케줄러가 1/16박마다 _onStep ─▶ 이 구간에 시작하는 클립과 노트를 예약
```

2. 언제 쓰나

쓰는 곳	하는 일
`sequencer-audio.js` (296)	재생 버튼의 `onPlay`가 `start`를, `onPause`가 `pause`를 부릅니다.
믹서 갱신 타이머	재생 중 150ms마다 `refreshMixer`를 불러 볼륨, 뮤트, 솔로, 이펙트 변경을 소리에 반영합니다.
미터	`meterLevels`로 트랙의 소리 크기를 읽습니다.

3. 코드

3.1 처음 만들기

```
_ensure() {
  if (this.engine) return;
  const self = this;
  const deps = this.deps;
  this.engine = new AudioEngine({
    context: deps.context,
    destination: deps.destination,
    baseBpm: deps.getBpm(),
    changes: deps.getTempoChanges ? deps.getTempoChanges() : [],
```

```
    metronomeEnabled: deps.getMetronome ? deps.getMetronome() : false,
    onStep: function (tick, time) { self._onStep(tick, time); },
  });
  this.mixer = new MixerGraph(this.engine.context, this.engine.destination);
  this.player = new ClipPlayer(
    this.engine.context,
    function (trackId) { return self.mixer.trackInput(trackId); },
    function (sampleId) { return self.buffers[String(sampleId)] || null; },
  );
  this.effects = new EffectRack(
    this.engine.context,
    this.mixer,
    deps.getEffects ? deps.getEffects : function () { return []; },
    function () { return deps.getBpm(); },
  );
  this.rack = new InstrumentRack(
    this.engine.context,
    function (trackId) { return self.mixer.trackInput(trackId); },
    deps.getInstruments ? deps.getInstruments : function () { return []; },
  );
  this.notePlayer = new NotePlayer(this.engine.context, function (trackId, pitch, velocity, when, dur) {
    return self.rack.playNote(trackId, pitch, velocity, when, dur);
  });
}
```

3.2 샘플 불러오기

```
_loadSample(sampleId) {
  const key = String(sampleId);
  if (this.buffers[key]) return Promise.resolve(this.buffers[key]);
  if (this.loading[key]) return this.loading[key];
  const self = this;
  const ctx = this.engine.context;
  this.loading[key] = this.deps.loadSampleBytes(sampleId).then(function (bytes) {
    return ctx.decodeAudioData(bytes);
  }).then(function (buffer) {
    self.buffers[key] = buffer;
    return buffer;
  }).catch(function (err) {
    delete self.loading[key];
    if (typeof console !== 'undefined') console.warn('[audio] 샘플 ' + key + ' 을(를) 불러오지 못했습니
다', err);
    return null;
  });
  return this.loading[key];
}
```

3.3 재생 시작

```
async start(fromTick) {
  this._ensure();
  const self = this;
  const deps = this.deps;
  this.clips = deps.getClips();
  this.noteEvents = expandNotes(deps.getMidiClips ? deps.getMidiClips() : []);
```

```
  await Promise.all(this.clips.filter(function (c) { return c.sample_id != null; })
    .map(function (c) { return self._loadSample(c.sample_id); }));

  this.mixer.applyTracks(deps.getTracks());
  this.effects.sync(deps.getTracks());
  this.engine.setTempo(deps.getBpm(), deps.getTempoChanges ? deps.getTempoChanges() : []);
  if (deps.getMetronome) this.engine.setMetronome(deps.getMetronome());
  const loop = deps.getLoop ? deps.getLoop() : null;
  this.engine.setLoop(loop ? loop.start : 0, loop ? loop.end : 0, !!loop);

  this.player.stopAll();
  this.notePlayer.stopAll();
  this.lastTick = null;
  this.engine.start({ fromTick: fromTick });
  this.playing = true;
  // 시작 위치에 걸쳐 있는 클립은 중간부터 바로 울림
  const t = this.engine.transport;
  clipsCovering(this.clips, t.originTick).forEach(function (c) {
    self.player.schedule(c, t.originTick, t.originTime, self.engine.transport.baseBpm, self.engine.trans
port.changes);
  });
}
```

3.4 한 단계마다 예약

```
_onStep(tick, time) {
  const t = this.engine.transport;
  if (this.lastTick != null && tick < this.lastTick) {
    // 루프로 되감김: 이전 구간 소리는 이 시각에 끊고, 되감긴 위치에 걸친 클립을 다시 시작
    this.player.stopAll(time);
    this.notePlayer.stopAll(time);
    const self = this;
    clipsCovering(this.clips, tick).forEach(function (c) {
      self.player.schedule(c, tick, time, t.baseBpm, t.changes);
    });
  }
  this.lastTick = tick;
  const self2 = this;
  clipsStartingIn(this.clips, tick, tick + t.stepTicks).forEach(function (c) {
    self2.player.schedule(c, tick, time, t.baseBpm, t.changes);
  });
  notesStartingIn(this.noteEvents, tick, tick + t.stepTicks).forEach(function (ev) {
    self2.notePlayer.schedule(ev, tick, time, t.baseBpm, t.changes);
  });
}
```

3.5 재생 중 갱신, 미터, 정지

```
refreshMixer() {
  if (!this.mixer) return;
  this.mixer.applyTracks(this.deps.getTracks());
  this.effects.sync(this.deps.getTracks());
}
```

```
meterLevels(trackId) {
  if (!this.mixer) return null;
  const tracks = this.deps.getTracks();
  const track = tracks.filter(function (t) { return String(t.track_id) === String(trackId); })[0];
  return this.mixer.meterFor(track && track.track_type === 'MASTER' ? 'master' : trackId).read();
}
```

```
pause() {
  if (!this.engine) return;
  this.engine.pause();
  this.player.stopAll();
  this.notePlayer.stopAll();
  this.playing = false;
}
```

```
stop() {
  if (!this.engine) return;
  this.engine.stop();
  this.player.stopAll();
  this.notePlayer.stopAll();
  this.effects.flush();
  this.playing = false;
}
```

4. 최신 문법으로 쓰면

```
// _loadSample 을 async/await 로 쓴 모양
async _loadSample(sampleId) {
  const key = String(sampleId);
  if (this.buffers[key]) return this.buffers[key];
  try {
    const bytes = await this.deps.loadSampleBytes(sampleId);
    return (this.buffers[key] = await this.engine.context.decodeAudioData(bytes));
  } catch (err) {
    console.warn(`[audio] 샘플 ${key} 을(를) 불러오지 못했습니다`, err);
    return null;
  }
}
```

전통 문법 (실제 코드)	최신 문법
`.then(...).catch(...)` 연결	`async/await` 와 `try/catch`
`'[audio] 샘플 ' + key + ' 을(를)'`	템플릿 문자열 `` `...${key}...` ``
`function (c) { return c.sample_id != null; }`	`(c) => c.sample_id != null`
`deps.getMidiClips ? deps.getMidiClips() : []`	`deps.getMidiClips?.() ?? []`

5. 왜 이렇게

선택	이유
엔진, 믹서를 처음 한 번만 만듭니다 (`_ensure`)	브라우저는 사용자 조작 안에서 만든 `AudioContext`만 소리를 내게 합니다. 재생 버튼을 누를 때 만들면 이 조건을 지킵니다.
재생 전에 샘플을 모두 내려받아 디코딩합니다	재생 중에 받으면 처음 몇 초가 비어서 들립니다. 그래서 `start`가 `async`이고, 모든 샘플이 준비된 뒤에 엔진을 시작합니다.
같은 샘플은 한 번만 디코딩합니다	번호별로 버퍼와 진행 중인 약속(`loading`)을 담아 두어서 같은 클립이 여럿이어도 한 번만 받습니다.
재생 시작 시점의 클립을 스냅샷으로 씁니다	재생 중에 클립을 옮겨도 이미 예약한 소리와 어긋나지 않습니다. 옮긴 결과는 다음 재생부터 들립니다.
루프 되감기를 `tick < lastTick`으로 감지합니다	스케줄러가 루프 끝에서 되감으면 tick이 줄어듭니다. 이때 이전 구간의 소리를 되감기 시각에 끊습니다.
정지에서 이펙트를 `flush`합니다	딜레이와 리버브의 꼬리가 다음 재생에 섞이지 않게 합니다.
샘플을 못 받아도 재생은 계속합니다	실패한 샘플만 건너뛰고 경고를 남깁니다 (`null` 반환).

6. 시험

- `docs/handoff/s1-11a_tests/ui.mjs`는 재생, 뮤트, 해제, 솔로, 정지를 마스터 출력의 피크로 확인합니다. 10건 모두 통과했습니다.
- `docs/handoff/s1-11a_tests/ui_midi.mjs`는 같은 세션이 MIDI 노트도 재생하는지 확인합니다.
- 시험 중 발견한 일: 예시 화면의 재생선이 12.5마디에서 시작해서 0마디의 클립을 지나쳤고, 1초 샘플은 측정 전에 끝났습니다. 시험이 ■를 눌러 재생선을 0으로 되돌리고 8초 샘플을 쓰도록 고쳤습니다. 소리 코드의 결함이 아니었습니다.

7. 알려진 한계

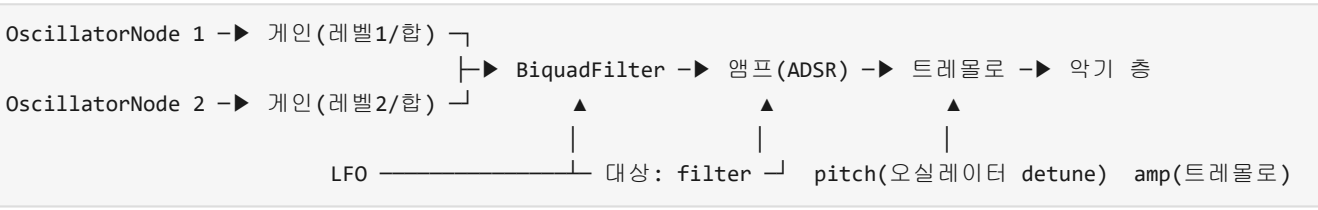
한계	설명
재생 중 재생선 이동이 소리에 반영되지 않습니다	정지 후 다시 재생해야 합니다.
곡 중간 템포 변경이 소리에 반영되지 않습니다	상단 BPM 값만 씁니다.
노트 중간에서 시작하면 그 노트는 울리지 않습니다	재생 위치 이후에 시작하는 노트만 울립니다.

JavaScript 291 — Subtractive Synth (`audio/synth.js`)

항목	내용
파일	`studionx/frontend/static/js/audio/synth.js`
작업	S1-11a 재생 오디오 그래프: 감산 신스가 노트 하나를 소리로 냄 (FR-242)
작성	Claude (AI), 2026-10-19, 사용자 검토
이전, 다음	이전 290 재생 세션, 다음 292 노트 재생과 악기 층

1. 하는 일

노트 하나를 "오실레이터 2개, 필터, 앰프 엔벨로프(ADSR), LFO" 그래프로 만들어 울립니다. 파라미터는 `plugins.param_schema`의 key를 그대로 읽습니다. 계산 부분은 순수 함수로 분리했습니다.



2. 언제 쓰나

쓰는 곳	하는 일
`InstrumentRack.playNote` (292)	트랙에 붙은 `synth_sub` 층마다 노트가 올 때 `playSynthNote`를 부릅니다.
WAV 출력 (S1-11b 예정)	같은 함수를 `OfflineAudioContext`에서 불러 노트를 렌더링합니다.

3. 코드

3.1 순수 함수

```
export function noteToFreq(pitch) {
  return 440 * Math.pow(2, (Number(pitch) - 69) / 12);
}
```

```
export function resolveParams(schema, params) {
  const out = {};
  (schema || []).forEach(function (item) {
    let value = item.default;
    if (params && params[item.key] !== undefined && params[item.key] !== null) value = params[item.key];
    if (typeof item.min === 'number' && typeof item.max === 'number' && typeof value === 'number') {
      value = Math.max(item.min, Math.min(item.max, value));
    }
    out[item.key] = value;
  });
  return out;
}
```

```
export function oscType(wave) {
  if (wave === 'saw') return 'sawtooth';
  if (wave === 'sine' || wave === 'triangle' || wave === 'square') return wave;
  return 'sawtooth';
}
```

```
export function envelopeLevel(t, a, d, s, peak) {
  if (t <= 0) return 0;
  if (a > 0 && t < a) return peak * (t / a);
  const td = t - a;
  if (d > 0 && td < d) return peak * (1 - (1 - s) * (td / d));
  return peak * s;
}
```

3.2 playSynthNote

```
export function playSynthNote(ctx, dest, p, pitch, velocity, when, duration) {
  const freq = noteToFreq(pitch);
  const peak = 0.3 * Math.max(0, Math.min(127, velocity)) / 127;
  const a = Number(p.env_attack) || 0;
  const d = Number(p.env_decay) || 0;
  const s = p.env_sustain != null ? Number(p.env_sustain) : 1;
  const r = Math.max(0.005, Number(p.env_release) || 0);
  const dur = Math.max(0.01, duration);

  // 앰프 엔벨로프
  const amp = ctx.createGain();
  amp.gain.setValueAtTime(0, when);
  if (a > 0) amp.gain.linearRampToValueAtTime(peak, when + a);
  else amp.gain.setValueAtTime(peak, when);
  if (d > 0 && dur > a) amp.gain.linearRampToValueAtTime(peak * s, when + Math.min(dur, a + d));
  const atRelease = envelopeLevel(dur, a, d, s, peak);
  amp.gain.setValueAtTime(atRelease, when + dur);
  amp.gain.linearRampToValueAtTime(0, when + dur + r);
  const end = when + dur + r + 0.02;

  // 필터
  const filter = ctx.createBiquadFilter();
  filter.type = p.filter_type === 'highpass' || p.filter_type === 'bandpass' ? p.filter_type : 'lowpas
s';
  filter.frequency.value = Math.max(20, Math.min(20000, Number(p.cutoff) || 1200));
  filter.Q.value = 0.5 + Math.max(0, Math.min(1, Number(p.resonance) || 0)) * 15;

  // 트레몰로(LFO → 앰프)는 앰프 뒤에 따로 두어 엔벨로프와 곱해지게 함
  const trem = ctx.createGain();
  filter.connect(amp);
  amp.connect(trem);
  trem.connect(dest);

  // 오실레이터 2개 (레벨은 합이 1이 되게 나눠 소리가 겹쳐도 커지지 않게 함)
  const l1 = Math.max(0, Number(p.osc1_level) || 0);
  const l2 = Math.max(0, Number(p.osc2_level) || 0);
  const sum = l1 + l2 > 0 ? l1 + l2 : 1;
  const oscs = [];
  [[l1, p.osc1_wave, Number(p.osc1_detune) || 0, 0], [l2, p.osc2_wave, Number(p.osc2_detune) || 0, Numbe
```

```
r(p.osc2_semi) || 0]]
    .forEach(function (cfg) {
      if (!(cfg[0] > 0)) return;
      const osc = ctx.createOscillator();
      osc.type = oscType(cfg[1]);
      osc.frequency.value = freq * Math.pow(2, cfg[3] / 12);
      osc.detune.value = cfg[2];
      const g = ctx.createGain();
      g.gain.value = cfg[0] / sum;
      osc.connect(g);
      g.connect(filter);
      osc.start(when);
      osc.stop(end);
      oscs.push(osc);
    });

  // LFO (깊이 0 이거나 대상이 off 면 만들지 않음)
  const depth = Math.max(0, Math.min(1, Number(p.lfo_depth) || 0));
  const target = p.lfo_target;
  const extra = [];
  if (depth > 0 && (target === 'pitch' || target === 'filter' || target === 'amp')) {
    const lfo = ctx.createOscillator();
    lfo.frequency.value = Math.max(0.01, Number(p.lfo_rate) || 1);
    const lg = ctx.createGain();
    lfo.connect(lg);
    if (target === 'pitch') {
      lg.gain.value = depth * 100;                       // 최대 ±100 cent
      oscs.forEach(function (o) { lg.connect(o.detune); });
    } else if (target === 'filter') {
      lg.gain.value = depth * filter.frequency.value;   // 컷오프의 ±(깊이 × 컷오프)
      lg.connect(filter.frequency);
    } else {
      trem.gain.value = 1 - depth / 2;                  // 1 - 깊이/2 를 중심으로 ±깊이/2 만큼 흔듦
      lg.gain.value = depth / 2;
      lg.connect(trem.gain);
    }
    lfo.start(when);
    lfo.stop(end);
    extra.push(lfo);
  }

  return {
    // 정지 · 루프 되감기 때 소리를 끊음 (뚝 끊기는 소리를 줄이려고 아주 짧은 릴리스를 줌)
    stop: function (at) {
      const t = Math.max(at, ctx.currentTime);
      try { amp.gain.cancelScheduledValues(t); amp.gain.setTargetAtTime(0, t, 0.01); } catch (e) { /* 무
시 */ }
      oscs.concat(extra).forEach(function (o) { try { o.stop(t + 0.08); } catch (e) { /* 이미 끝남 */ }
});
    },
  };
}
```

4. 최신 문법으로 쓰면

```
// resolveParams 를 구조 분해와 nullish 병합으로 쓴 모양
export const resolveParams = (schema = [], params = null) =>
  Object.fromEntries(schema.map(({ key, min, max, default: def }) => {
    let value = params?.[key] ?? def;
    if (typeof min === 'number' && typeof max === 'number' && typeof value === 'number') {
      value = Math.max(min, Math.min(max, value));
    }
    return [key, value];
  }));
```

전통 문법 (실제 코드)	최신 문법
`params && params[item.key] !== undefined && params[item.key] !== null`	`params?.[item.key] ?? item.default`
`forEach` 로 객체에 담기	`Object.fromEntries(map(...))`
`item.default`	구조 분해 `{ default: def }`

5. 왜 이렇게

선택	이유
저장된 값을 `param_schema`의 범위로 자릅니다	잘못 저장된 값(예: 음수 어택)이 소리를 망가뜨리지 않게 합니다.
두 오실레이터의 레벨을 합으로 나눕니다	레벨을 둘 다 1로 두어도 소리가 두 배로 커지지 않습니다. 합이 1이 되게 정규화합니다.
릴리스 직전 값을 계산합니다 (`envelopeLevel`)	노트가 어택이나 디케이 중에 끝나도, 그 순간의 값에서 0으로 이어져야 "탁" 하는 잡음이 없습니다.
LFO 대상을 세 가지로 나눴습니다	피치는 `detune`(최대 ±100센트), 필터는 컷오프(±깊이 곱하기 컷오프), 앰프는 트레몰로(1 - 깊이/2를 중심으로 ±깊이/2)를 흔듭니다.
트레몰로를 앰프 뒤 별도 게인으로 뒀습니다	엔벨로프와 곱해져야 해서 같은 게인에 이으면 서로 덮어씁니다.
오실레이터는 노트마다 새로 만듭니다	한 번 `stop`한 오실레이터는 다시 쓸 수 없습니다. 가볍게 만들 수 있게 설계된 노드입니다.
`stop`은 짧은 릴리스로 끊습니다	재생을 멈출 때 소리가 갑자기 끊기면 클릭이 납니다. 10ms 시간 상수로 줄이고 80ms 뒤 멈춥니다.
동시 발음 수에 제한이 없습니다	지금은 단순함을 우선했습니다. 노트가 매우 많아지면 제한을 둘 수 있습니다.

6. 시험

- `docs/handoff/s1-11a_tests/synth_notes_unit.mjs`는 다음을 확인합니다. A4는 440Hz이고 A5는 880Hz입니다. 범위를 벗어난 컷오프는 20000으로 잘립니다. 스키마에 없는 키는 버려지고, `null` 값은 기본값을 씁니다. 엔벨로프는 어택 중 절반, 디케이 중 75%, 서스테인에서 50%입니다.
- `docs/handoff/s1-11a_tests/ui_midi.mjs`는 실제 화면에서 A4 노트가 소리로 나오는지 확인합니다. 피크가 0.05보다 크고 0.5보다 작고, 뮤트하면 무음이 됩니다.
- 시험 중 발견한 일: 처음에는 소리가 거의 0이었는데, 그 트랙의 기존 Synth에 컷오프 20Hz가 저장되어 있었고 트랙 볼륨도 -21.5dB였기 때문입니다. 코드는 정상이었고, 시험이 기본값의 새 Synth를 붙이도록 고

쳤습니다.

JavaScript 292 — MIDI 노트 재생과 악기 층 (`audio/notes.js`, `audio/instruments.js`)

항목	내용
파일	`studionx/frontend/static/js/audio/notes.js`, `studionx/frontend/static/js/audio/instruments.js`
작업	S1-11a 재생 오디오 그래프: MIDI 클립의 노트를 트랙의 악기 층으로 연주 (FR-205, FR-233, FR-241)
작성	Claude (AI), 2026-10-19, 사용자 검토
이전, 다음	이전 291 Synth, 다음 293 이펙트

1. 하는 일

두 파일이 함께 노트를 소리로 바꿉니다. `notes.js`는 "어느 노트가 언제 시작하는지"를 계산하고, `instruments.js`는 "그 노트를 트랙의 어떤 악기로 연주하는지"를 맡습니다.

```
MIDI 클립 { start_tick, length_tick, offset_tick, notes[] }
      │  expandNotes        노트를 타임라인 절대 위치로 펼칩니다 (음소거 제외, 클립 끝에서 자름)
      ▼
notesStartingIn(구간)      이번 단계에서 새로 시작하는 노트를 고릅니다
      ▼
planNote                   when(시각), durationSec(길이)로 바꿉니다
      ▼
NotePlayer.schedule ─▶ InstrumentRack.playNote ─▶ 트랙의 악기 층마다 playSynthNote
                                                  (층: 볼륨, 팬 노드 ─▶ 트랙 입구)
```

2. 언제 쓰나

쓰는 곳	하는 일
`PlaybackSession.start`	재생을 시작할 때 MIDI 클립 전체를 `expandNotes`로 한 번 펼쳐 둡니다.
`PlaybackSession._onStep`	단계마다 `notesStartingIn`으로 새 노트를 골라 `NotePlayer.schedule`로 예약합니다.
정지, 일시정지, 루프 되감기	`NotePlayer.stopAll`이 울리는 노트를 모두 끕니다.

3. 코드

3.1 노트 계산 (`notes.js`)

```
export function expandNotes(clips) {
  const out = [];
  (clips || []).forEach(function (clip) {
    const offset = clip.offset_tick || 0;
    const clipEnd = clip.start_tick + clip.length_tick;
    (clip.notes || []).forEach(function (n) {
      if (n.is_muted) return;
      if (n.start_tick < offset) return;
      const abs = clip.start_tick + (n.start_tick - offset);
```

```
      if (abs >= clipEnd) return;
      out.push({
        track_id: clip.track_id,
        pitch: n.pitch,
        velocity: n.velocity != null ? n.velocity : 100,
        abs_start: abs,
        length_tick: Math.max(1, Math.min(n.length_tick, clipEnd - abs)),
      });
    });
  });
  out.sort(function (a, b) { return a.abs_start - b.abs_start; });
  return out;
}
```

```
export function notesStartingIn(events, fromTick, toTick) {
  return (events || []).filter(function (e) { return e.abs_start >= fromTick && e.abs_start < toTick;
});
}
```

```
export function planNote(ev, nowTick, nowTime, baseBpm, changes) {
  const sec = function (a, b) {
    return ticksToSecondsWithChanges(b, baseBpm, changes) - ticksToSecondsWithChanges(a, baseBpm, change
s);
  };
  return { when: nowTime + sec(nowTick, ev.abs_start), durationSec: sec(ev.abs_start, ev.abs_start + ev.
length_tick) };
}
```

3.2 노트 재생기 (`notes.js`)

```
schedule(ev, nowTick, nowTime, baseBpm, changes) {
  const plan = planNote(ev, nowTick, nowTime, baseBpm, changes);
  if (!(plan.durationSec > 0)) return 0;
  const voices = this.playOnTrack(ev.track_id, ev.pitch, ev.velocity, Math.max(plan.when, this.context.c
urrentTime), plan.durationSec);
  const self = this;
  (voices || []).forEach(function (v) {
    self.active.push({ voice: v, end: plan.when + plan.durationSec + 6 });   // 릴리스 여유 포함
  });
  this._prune();
  return (voices || []).length;
}
```

```
stopAll(atTime) {
  const t = atTime != null ? atTime : this.context.currentTime;
  this.active.forEach(function (a) { try { a.voice.stop(t); } catch (e) { /* 무시 */ } });
  this.active = [];
}
```

3.3 악기 층 (`instruments.js`)

```
_layer(trackId, inst) {
  const key = String(trackId) + ':' + String(inst.id);
```

```
  const ctx = this.context;
  let layer = this.layers[key];
  if (!layer) {
    const gain = createStereoGain(ctx);
    const pan = typeof ctx.createStereoPanner === 'function' ? ctx.createStereoPanner() : null;
    if (pan) { gain.connect(pan); pan.connect(this.trackInput(trackId)); }
    else gain.connect(this.trackInput(trackId));
    layer = { gain: gain, pan: pan };
    this.layers[key] = layer;
  }
  layer.gain.gain.value = layerGain(inst.volume_db);
  if (layer.pan) layer.pan.pan.value = Math.max(-1, Math.min(1, Number(inst.pan) || 0));
  return layer;
}
```

```
playNote(trackId, pitch, velocity, when, durationSec) {
  const out = [];
  const self = this;
  (this.getInstruments(trackId) || []).forEach(function (inst) {
    if (inst.is_enabled === false) return;
    if (PLAYABLE_CODES.indexOf(inst.plugin_code) < 0) return;
    const layer = self._layer(trackId, inst);
    const params = resolveParams(inst.schema, inst.params);
    out.push(playSynthNote(self.context, layer.gain, params, pitch, velocity, when, durationSec));
  });
  return out;
}
```

4. 최신 문법으로 쓰면

```
// expandNotes 를 구조 분해와 flatMap 으로 쓴 모양
export const expandNotes = (clips = []) =>
  clips
    .flatMap(({ track_id, start_tick, length_tick, offset_tick = 0, notes = [] }) =>
      notes
        .filter((n) => !n.is_muted && n.start_tick >= offset_tick
          && start_tick + (n.start_tick - offset_tick) < start_tick + length_tick)
        .map((n) => {
          const abs = start_tick + (n.start_tick - offset_tick);
          return {
            track_id,
            pitch: n.pitch,
            velocity: n.velocity ?? 100,
            abs_start: abs,
            length_tick: Math.max(1, Math.min(n.length_tick, start_tick + length_tick - abs)),
          };
        }))
    .sort((a, b) => a.abs_start - b.abs_start);
```

전통 문법 (실제 코드)	최신 문법
`forEach` 안에서 배열에 `push`	`flatMap`, `filter`, `map`의 연결
`n.velocity != null ? n.velocity : 100`	`n.velocity ?? 100`

전통 문법 (실제 코드)	최신 문법
`clip.offset_tick || 0`	구조 분해 기본값 `offset_tick = 0`

5. 왜 이렇게

선택	이유
재생 시작 때 한 번만 펼칩니다	노트는 수천 개일 수 있어서, 단계마다(1/16박) 전부 훑으면 느립니다. `abs_start` 순으로 정렬해 두고 구간만 걸러 냅니다.
클립의 `offset_tick`을 오디오 클립과 같은 뜻으로 씁니다	클립을 왼쪽에서 잘라 내면 `offset_tick`이 늘어납니다. 그 앞의 노트는 빼고, 남은 노트는 클립 시작에서 당겨 놓습니다.
클립 밖으로 나가는 노트는 클립 끝에서 자릅니다	클립을 줄였는데 노트가 클립 밖까지 울리면 안 됩니다.
음소거된 노트는 펼칠 때 뺍니다	재생 중에는 걸러 낼 필요가 없어서 가볍습니다.
악기 층마다 볼륨과 팬을 따로 둡니다	한 트랙에 악기가 여러 개일 때(레이어) 층마다 소리 크기와 좌우가 다릅니다. 층 노드는 한 번 만들어 재사용합니다.
Sampler는 MIDI로 소리가 나지 않습니다	시드의 Sampler에는 샘플을 고르는 칸이 없어서, 노트를 연주할 소리를 알 수 없습니다. 목업을 본 뒤 결정하기로 했습니다 (`PLAYABLE_CODES`에 `synth_sub`만 있음).
울리는 소리 목록을 정리합니다 (`_prune`)	끝난 소리를 계속 담아 두면 목록이 끝없이 자랍니다. 6초 지난 것은 뺍니다.
노트가 시작된 뒤에 재생을 시작하면 그 노트는 울리지 않습니다	단순함을 우선했습니다. 필요하면 시작 위치에 걸친 노트를 중간부터 울리게 확장할 수 있습니다.

6. 시험

- `docs/handoff/s1-11a_tests/synth_notes_unit.mjs`는 다음을 확인합니다. 음소거 노트는 빠지고 클립 밖 노트도 빠집니다. 클립 끝에서 길이가 잘리고, `velocity`가 없으면 100이 됩니다. `offset_tick`이 있으면 앞 노트가 빠지고 나머지가 당겨집니다. 구간 선택은 시작은 포함하고 끝은 포함하지 않습니다. 120BPM에서 960틱이 0.5초입니다.
- `docs/handoff/s1-11a_tests/ui_midi.mjs`는 실제 화면에서 MIDI 클립의 노트가 Synth 소리로 나오는지 확인합니다.

JavaScript 293 — 이펙트 6종 (`audio/effects.js`)

항목	내용
파일	`studionx/frontend/static/js/audio/effects.js`
작업	S1-11a 재생 오디오 그래프: Gain, EQ 3-Band, Compressor, Limiter, Delay, Reverb를 Web Audio 노드로 구현 (FR-243, FR-247)
작성	Claude (AI), 2026-10-19, 사용자 검토
이전, 다음	이전 292 노트와 악기 층, 다음 294 이펙트 체인 관리

1. 하는 일

시드의 이펙트 6종을 Web Audio 기본 노드의 조합으로 만듭니다. 모든 이펙트는 같은 모양입니다.

```
{ input, output, update(params), dispose() }

input  ─▶ (이펙트 내부 노드들) ─▶ output
update  노브가 바뀔 때 새 값을 소리에 반영합니다
dispose 노드 연결을 끊어 정리합니다
```

이펙트	코드	노드 조합
Gain / Utility	`gain`	스테레오 게인, `StereoPannerNode`, 위상 반전은 게인을 음수로
EQ 3-Band	`eq3`	`BiquadFilter` 3개 (lowshelf, peaking, highshelf)
Compressor	`compressor`	`DynamicsCompressorNode`, 메이크업 게인
Limiter	`limiter`	`DynamicsCompressorNode`(레이시오 20, 니 0), 보정 게인
Delay	`delay`	`DelayNode`, 되먹임 게인, 톤 필터, 첫 반복 보정 딜레이
Reverb	`reverb`	프리딜레이, `ConvolverNode`(코드로 만든 임펄스), 원음과 섞기

2. 언제 쓰나

쓰는 곳	하는 일
`EffectRack` (294)	트랙에 붙은 이펙트마다 `createEffect`로 만들고, 노브가 바뀌면 `update`를 부릅니다.
WAV 출력 (S1-11b 예정)	같은 이펙트를 `OfflineAudioContext`에 만들어 렌더링합니다.

3. 코드

3.1 순수 함수

```
export function delaySeconds(p, bpm) {
  let sec;
  if (p.sync) {
    const beats = DIVISION_BEATS[p.division] != null ? DIVISION_BEATS[p.division] : 0.5;
    sec = beats * 60 / (bpm > 0 ? bpm : 120);
  } else {
```

```
    sec = Number(p.time) || 0.35;
  }
  return Math.max(0.001, Math.min(MAX_DELAY_SECONDS, sec));
}
```

```
export function makeImpulse(sampleRate, decay, damping, channel) {
  const length = Math.max(1, Math.floor(sampleRate * Math.max(0.1, decay)));
  const out = new Float32Array(length);
  let seed = 12345 + (channel || 0) * 7919;
  const coef = 1 - Math.max(0, Math.min(1, damping)) * 0.9;   // 1 = 그대로, 0.1 = 크게 둔하게
  let lp = 0;
  for (let i = 0; i < length; i++) {
    seed = (Math.imul(seed, 1664525) + 1013904223) >>> 0;
    const noise = (seed / 4294967296) * 2 - 1;
    const envelope = Math.pow(0.001, i / length);   // 끝에서 -60dB
    lp = lp + coef * (noise - lp);
    out[i] = lp * envelope;
  }
  return out;
}
```

```
export function compressorAutoMakeupDb(threshold, ratio) {
  const r = Math.max(1, ratio);
  const gainDb = Math.min(0, threshold) * (1 - 1 / r);   // 0dB 입력일 때의 감쇠(dB, 음수)
  return -0.6 * gainDb;
}
```

3.2 Gain 과 EQ

```
function createGainFx(ctx) {
  const gain = createStereoGain(ctx);
  const pan = typeof ctx.createStereoPanner === 'function' ? ctx.createStereoPanner() : null;
  if (pan) gain.connect(pan);
  return {
    input: gain,
    output: pan || gain,
    update: function (p) {
      const g = dbToGain(p.gain);
      gain.gain.value = p.phase_invert ? -g : g;
      if (pan) pan.pan.value = Math.max(-1, Math.min(1, Number(p.pan) || 0));
    },
    dispose: function () { drop(gain); if (pan) drop(pan); },
  };
}
```

```
function createEq3Fx(ctx) {
  const low = ctx.createBiquadFilter(); low.type = 'lowshelf';
  const mid = ctx.createBiquadFilter(); mid.type = 'peaking';
  const high = ctx.createBiquadFilter(); high.type = 'highshelf';
  low.connect(mid); mid.connect(high);
  return {
    input: low,
    output: high,
    update: function (p) {
```

```
      low.frequency.value = Number(p.low_freq) || 200;
      low.gain.value = Number(p.low_gain) || 0;
      mid.frequency.value = Number(p.mid_freq) || 1000;
      mid.gain.value = Number(p.mid_gain) || 0;
      mid.Q.value = Number(p.mid_q) || 0.7;
      high.frequency.value = Number(p.high_freq) || 6000;
      high.gain.value = Number(p.high_gain) || 0;
    },
    dispose: function () { drop(low); drop(mid); drop(high); },
  };
}
```

3.3 Compressor 와 Limiter

```
function createCompressorFx(ctx) {
  const comp = ctx.createDynamicsCompressor();
  const makeup = ctx.createGain();
  comp.connect(makeup);
  return {
    input: comp,
    output: makeup,
    update: function (p) {
      const threshold = p.threshold != null ? Number(p.threshold) : -20;
      const ratio = Math.max(1, Math.min(20, Number(p.ratio) || 4));
      comp.threshold.value = threshold;
      comp.ratio.value = ratio;
      comp.attack.value = Math.max(0, Math.min(1, Number(p.attack) || 0.01));
      comp.release.value = Math.max(0, Math.min(1, Number(p.release) || 0.2));
      comp.knee.value = Math.max(0, Math.min(40, p.knee != null ? Number(p.knee) : 10));
      // 브라우저가 자동으로 더하는 메이크업을 덜어 내고, 사용자가 정한 메이크업만 더함
      makeup.gain.value = dbToGain((Number(p.makeup) || 0) - compressorAutoMakeupDb(threshold, ratio));
    },
    dispose: function () { drop(comp); drop(makeup); },
  };
}
```

```
function createLimiterFx(ctx) {
  const comp = ctx.createDynamicsCompressor();
  const trim = ctx.createGain();
  comp.ratio.value = 20;
  comp.knee.value = 0;
  comp.attack.value = 0.001;
  comp.connect(trim);
  return {
    input: comp,
    output: trim,
    update: function (p) {
      const ceiling = Math.max(-100, Math.min(0, p.ceiling != null ? Number(p.ceiling) : -0.3));
      comp.threshold.value = ceiling;
      comp.release.value = Math.max(0.01, Math.min(1, Number(p.release) || 0.1));
      trim.gain.value = dbToGain(-compressorAutoMakeupDb(ceiling, 20));   // 브라우저의 자동 메이크업을
덜어 내야 천장을 넘지 않음
    },
    dispose: function () { drop(comp); drop(trim); },
```

```
  };
}
```

3.4 Delay

```
function createDelayFx(ctx, getBpm) {
  const input = ctx.createGain();
  const output = ctx.createGain();
  const delay = ctx.createDelay(MAX_DELAY_SECONDS);
  const tone = ctx.createBiquadFilter(); tone.type = 'lowpass';
  const feedback = ctx.createGain();
  const tap = ctx.createDelay(0.1);      // 고리를 한 바퀴 돌 때 더 걸리는 128 샘플만큼 첫 반복을 늦춤 (아래 설명)
  const wet = ctx.createGain();
  input.connect(output);                 // 원음
  input.connect(delay);
  delay.connect(tone);
  tone.connect(feedback);
  feedback.connect(delay);               // 되먹임
  tone.connect(tap);
  tap.connect(wet);
  wet.connect(output);
  // 브라우저는 되먹임 고리를 한 바퀴 돌 때마다 128 샘플(한 번 처리 단위)을 더 걸리게 합니다.
  // 그래서 고리의 딜레이는 그만큼 줄이고, 첫 반복은 tap 으로 같은 양을 늦춰서 모든 반복이 정확히 time 간격이 되게 함
  return {
    input: input,
    output: output,
    update: function (p) {
      const q = QUANTUM / ctx.sampleRate;
      const time = Math.max(2 * q, delaySeconds(p, getBpm()));
      delay.delayTime.value = time - q;
      tap.delayTime.value = q;
      feedback.gain.value = Math.max(0, Math.min(0.95, Number(p.feedback) || 0));
      tone.frequency.value = Math.max(200, Math.min(20000, Number(p.tone) || 6000));
      wet.gain.value = Math.max(0, Math.min(1, p.mix != null ? Number(p.mix) : 0.3));
    },
    dispose: function () { [input, output, delay, tone, feedback, tap, wet].forEach(drop); },
  };
}
```

3.5 Reverb

```
function createReverbFx(ctx) {
  const input = ctx.createGain();
  const output = ctx.createGain();
  const dry = ctx.createGain();
  const wet = ctx.createGain();
  const pre = ctx.createDelay(0.5);
  const conv = ctx.createConvolver();
  input.connect(dry); dry.connect(output);
  input.connect(pre); pre.connect(conv); conv.connect(wet); wet.connect(output);
  let key = '';
  return {
    input: input,
```

```
    output: output,
    update: function (p) {
      const decay = Math.max(0.1, Math.min(10, Number(p.decay) || 2));
      const damping = Math.max(0, Math.min(1, p.damping != null ? Number(p.damping) : 0.5));
      const next = decay + ':' + damping;
      if (next !== key) {
        key = next;
        const sr = ctx.sampleRate;
        const buf = ctx.createBuffer(2, Math.max(1, Math.floor(sr * decay)), sr);
        buf.copyToChannel(makeImpulse(sr, decay, damping, 0), 0);
        buf.copyToChannel(makeImpulse(sr, decay, damping, 1), 1);
        conv.buffer = buf;
      }
      pre.delayTime.value = Math.max(0, Math.min(0.5, Number(p.pre_delay) || 0));
      const mix = Math.max(0, Math.min(1, p.mix != null ? Number(p.mix) : 0.3));
      dry.gain.value = Math.cos(mix * Math.PI / 2);
      wet.gain.value = Math.sin(mix * Math.PI / 2) * 0.5;   // 컨볼버는 소리를 크게 만들어서 절반으로
    },
    dispose: function () { [input, output, dry, wet, pre, conv].forEach(drop); },
  };
}
```

3.6 만들기

```
export function createEffect(ctx, code, params, getBpm) {
  const make = FACTORIES[code];
  if (!make) return null;
  const fx = make(ctx, getBpm || function () { return 120; });
  fx.update(params || {});
  return fx;
}
```

4. 최신 문법으로 쓰면

```
// delaySeconds 를 구조 분해와 nullish 병합으로 쓴 모양
const DIVISION_BEATS = { '1/4': 1, '1/8': 0.5, '1/8d': 0.75, '1/16': 0.25 };
export const delaySeconds = ({ sync, division, time }, bpm) => {
  const sec = sync
    ? (DIVISION_BEATS[division] ?? 0.5) * 60 / (bpm > 0 ? bpm : 120)
    : Number(time) || 0.35;
  return Math.max(0.001, Math.min(MAX_DELAY_SECONDS, sec));
};
```

전통 문법 (실제 코드)	최신 문법
`DIVISION_BEATS[p.division] != null ? ... : 0.5`	`DIVISION_BEATS[division] ?? 0.5`
`p.mix != null ? Number(p.mix) : 0.3`	`p.mix ?? 0.3` (숫자 변환은 별도)
객체를 `p` 로 받아 `p.sync`	구조 분해 `{ sync, division, time }`

5. 왜 이렇게

선택	이유
이펙트를 같은 모양 `{ input, output, update, dispose }`로 통일했습니다	체인에서 이펙트 종류를 몰라도 `output`을 다음 `input`에 잇기만 하면 됩니다. 이펙트를 더하기 쉬워집니다.
노브가 바뀌면 `update`만 부릅니다	노드를 다시 만들면 소리가 끊깁니다. 값만 바꾸면 소리가 이어집니다.
리버브 임펄스를 고정 시드 난수로 만듭니다	같은 설정이면 늘 같은 잔향이 나야 합니다. 실시간과 WAV 출력, 회귀 시험이 같은 소리를 내게 하려는 것입니다.
임펄스는 `decay`와 `damping`이 바뀔 때만 다시 만듭니다	임펄스 생성은 계산이 많아서 믹스나 프리딜레이를 돌릴 때마다 만들지 않습니다.
리버브 믹스를 등출력 곡선으로 섞습니다	원음은 cos, 잔향은 sin이라 중간 값에서 전체 소리 크기가 일정합니다. 잔향은 컨볼버가 키우는 만큼 절반으로 줄입니다.
컴프레서와 리미터의 자동 메이크업을 덜어냅니다	브라우저의 컴프레서는 압축한 만큼 스스로 키워서, 그대로 쓰면 리미터가 천장을 넘고 메이크업 노브가 이중으로 걸립니다. 계산해서 같은 양을 줄입니다.
리미터는 근사치입니다	기본 컴프레서(레이시오 20, 니 0)로 흉내 낸 것이라 정확한 피크 리미터는 아닙니다. 이후 WASM으로 교체합니다 (`engine='WASM'`).
딜레이 고리에서 128샘플을 뺍니다	브라우저는 되먹임 고리를 한 바퀴 돌 때마다 128샘플을 더 걸리게 합니다. 고리의 시간을 줄이고 첫 반복을 따로 늦춰서 모든 반복이 정확한 간격이 됩니다.
Gain 이펙트의 입구를 스테레오로 고정했습니다	모노 소리가 `StereoPannerNode`를 지나면 가운데에서 -3dB가 됩니다. 이것을 막아 Gain을 -12dB로 두면 정확히 -12dB가 됩니다.
딜레이 시간을 템포에서 계산합니다	`sync`가 켜지면 박 길이(1/4, 1/8, 점8분, 1/16)와 BPM으로 초를 구합니다. BPM이 바뀌어도 맞도록 `EffectRack`가 매번 다시 계산합니다.

6. 시험

- `docs/handoff/s1-11a_tests/effects_unit.mjs` (17건)는 순수 함수를 확인합니다. 딜레이 시간은 120BPM의 1/4가 0.5초이고 1/8이 0.25초이며 1/8 점음표가 0.375초입니다. 임펄스는 길이가 맞고 같은 입력이면 같은 결과이며 좌우가 다르고 시간이 지나면 줄어듭니다. 댐핑이 크면 고음이 줄어듭니다. 자동 메이크업은 천장 -12dB, 레이시오 20에서 6.84dB입니다.
- `docs/handoff/s1-11a_tests/effects_offline.mjs` (16건)는 `OfflineAudioContext`로 실제 소리를 계산해서 확인합니다.

이펙트	확인한 내용
Gain	-12dB에서 크기가 약 1/4이고, +6dB에서 약 2배입니다.
EQ	저음 -24dB에서 100Hz 소리가 30% 아래로 줄고, 중음 +12dB에서 1kHz 소리가 약 4배이며, 모두 0dB이면 그대로입니다.
Compressor	임계값 -40dB, 20대 1에서 소리가 절반 아래로 줄고, 메이크업 +12dB에서 약 4배가 됩니다.
Limiter	천장 -12dB에서 입력 피크 0.5가 0.4 아래로 줄어듭니다.
Delay	원음이 그대로이고, 0.1초와 0.2초에 반복이 오며(되먹임 0.5), 120BPM의 1/8은 0.25초에 반복합니다.
Reverb	믹스 1이면 꼬리가 남고, 믹스 0이면 꼬리가 없으며, 같은 설정이면 렌더링 결과가 같습니다.

- 시험이 처음에 4건 실패했고, 모두 Web Audio의 동작 특성에서 비롯되었습니다. 자세한 내용은 문서 83의 10절을 봅니다.

JavaScript 294 — 이펙트 체인 관리 (`audio/effect_rack.js`)

항목	내용
파일	`studionx/frontend/static/js/audio/effect_rack.js`
작업	S1-11a 재생 오디오 그래프: 트랙의 이펙트 목록을 읽어 체인을 만들고, 바뀌면 다시 잇고, 노브 값을 반영 (FR-243, FR-263)
작성	Claude (AI), 2026-10-19, 사용자 검토
이전, 다음	이전 293 이펙트, 다음 295 레벨 미터

1. 하는 일

화면이 주는 "트랙의 이펙트 목록"(슬롯 순서, 바이패스, 파라미터)을 실제 이펙트 노드 체인으로 맞춥니다. 화면은 목록만 주고, 소리 쪽 변경은 모두 여기서 일어납니다.

```
getEffects(트랙) ─▶ 바이패스와 모르는 종류를 거름 ─▶ slot_index 순서로 정렬
        │
        ├ 새 이펙트          createEffect 로 만듭니다
        ├ 값이 바뀐 이펙트    update 로 소리에 반영합니다 (노드를 다시 만들지 않음)
        ├ 순서나 구성이 바뀜  믹서의 체인을 끊고 다시 잇습니다
        └ 사라진 이펙트      dispose 로 정리합니다
```

2. 언제 쓰나

쓰는 곳	하는 일
`PlaybackSession.start`	재생을 시작할 때 `sync`를 불러 현재 이펙트 구성을 소리에 만듭니다.
`PlaybackSession.refreshMixer`	재생 중 150ms마다 `sync`를 불러 이펙트 추가, 삭제, 순서, 바이패스, 노브 변경을 소리에 반영합니다.
`PlaybackSession.stop`	`flush`로 이펙트를 비워서 딜레이와 리버브의 꼬리가 다음 재생에 섞이지 않게 합니다.

3. 코드

```
_syncTrack(key, trackId) {
  const items = (this.getEffects(trackId) || [])
    .filter(function (e) { return !e.is_bypassed && EFFECT_CODES.indexOf(e.plugin_code) >= 0; })
    .sort(function (a, b) { return a.slot_index - b.slot_index; });

  const self = this;
  const chain = [];
  items.forEach(function (e) {
    const params = resolveParams(e.schema, e.params);
    const sig = JSON.stringify(params);
    let entry = self.live[e.id];
    if (entry && entry.code !== e.plugin_code) { entry.fx.dispose(); entry = null; delete self.live[e.i
d]; }
    if (!entry) {
```

```
      entry = { fx: createEffect(self.context, e.plugin_code, params, self.getBpm), code: e.plugin_code,
sig: sig, track: key };
      self.live[e.id] = entry;
    } else if (entry.sig !== sig) {
      entry.fx.update(params);
      entry.sig = sig;
    } else if (e.plugin_code === 'delay') {
      entry.fx.update(params);    // 템포 동기 딜레이는 BPM 이 바뀌면 값이 달라져서 매번 다시 계산함
    }
    chain.push({ id: e.id, fx: entry.fx });
  });

  const signature = chain.map(function (c) { return c.id; }).join(',');
  if (this.chains[key] !== signature) {
    const list = chain.map(function (c) { return c.fx; });
    if (key === 'master') this.mixer.setMasterEffects(list);
    else this.mixer.setTrackEffects(trackId, list);
    this.chains[key] = signature;
  }

  // 목록에서 사라진 이펙트(삭제 · 종류 바뀜)는 정리
  const keep = {};
  chain.forEach(function (c) { keep[c.id] = true; });
  Object.keys(this.live).forEach(function (id) {
    if (self.live[id].track === key && !keep[id]) { self.live[id].fx.dispose(); delete self.live[id]; }
  });
}
```

```
sync(tracks) {
  const self = this;
  (tracks || []).forEach(function (t) {
    self._syncTrack(t.track_type === 'MASTER' ? 'master' : String(t.track_id), t.track_id);
  });
}
```

```
flush() {
  const self = this;
  Object.keys(this.chains).forEach(function (key) {
    if (key === 'master') self.mixer.setMasterEffects([]);
    else self.mixer.setTrackEffects(key, []);
  });
  Object.keys(this.live).forEach(function (id) { self.live[id].fx.dispose(); });
  this.live = {};
  this.chains = {};
}
```

4. 최신 문법으로 쓰면

```
// 체인 구성이 바뀌었는지 비교하는 부분을 최신 문법으로 쓴 모양
const signature = chain.map(({ id }) => id).join(',');
if (this.chains[key] !== signature) {
  const list = chain.map(({ fx }) => fx);
  key === 'master' ? this.mixer.setMasterEffects(list) : this.mixer.setTrackEffects(trackId, list);
```

```
  this.chains[key] = signature;
}
```

전통 문법 (실제 코드)	최신 문법
`function (c) { return c.id; }`	`({ id }) => id`
`if (key === 'master') ... else ...`	삼항 연산자
`Object.keys(obj).forEach(function (id) { ... })`	`for (const id of Object.keys(obj))`

5. 왜 이렇게

선택	이유
이펙트 번호를 키로 노드를 보관합니다 (`live`)	같은 이펙트의 노브만 바뀌었을 때 노드를 다시 만들지 않고 값만 바꿉니다. 재생이 끊기지 않습니다.
파라미터를 문자열로 만들어 비교합니다 (`sig`)	값이 안 바뀌었으면 아무것도 하지 않습니다. 150ms마다 불러도 가볍습니다.
체인 구성이 바뀔 때만 다시 잇습니다	연결을 끊고 잇는 순간 소리가 잠깐 끊길 수 있어서, 이펙트 목록이 실제로 달라졌을 때만 합니다.
딜레이는 매번 `update`를 부릅니다	템포 동기 딜레이는 BPM이 바뀌면 시간이 달라지는데, 파라미터 값은 그대로라 비교로는 알 수 없습니다.
바이패스한 이펙트는 체인에서 빼고 노드도 정리합니다	꺼진 이펙트가 소리에 영향을 주지 않게 하는 가장 단순한 방법입니다. 다시 켜면 새로 만들어서 같은 값으로 소리가 나지만, 이전의 딜레이와 리버브 꼬리는 이어지지 않습니다.
MASTER 트랙은 특별한 키 (`master`)를 씁니다	마스터 이펙트는 마스터 볼륨 뒤에 들어가서 일반 트랙과 다른 연결을 씁니다.
정지에서 `flush`합니다	이펙트를 비우고 다음 재생에서 새로 만듭니다. 이전 재생의 꼬리가 섞이는 것을 막습니다.
모르는 종류(Sampler 등)는 건너뜁니다	`EFFECT_CODES`에 없는 코드는 소리를 바꾸지 않고 지나갑니다.

6. 시험

- `docs/handoff/s1-11a_tests/ui_fx.mjs` (8건)는 실제 화면에서 다음을 확인합니다.

시험	결과
이펙트 없음	피크가 사인파 진폭 0.4와 같습니다.
트랙에 Gain -12dB	피크가 약 1/4(0.1)입니다.
바이패스	원래 소리(0.4)로 돌아옵니다.
바이패스를 풀고 Gain -24dB	피크가 약 1/16(0.025)입니다.
마스터 트랙에 Limiter 천장 -12dB	최종 출력의 피크가 0.3 아래로 줄어듭니다.

- 시험 데이터는 이펙트가 없는 시험 전용 오디오 트랙을 새로 만들어 쓰고, 끝나면 모두 지웁니다 (기존 트랙에는 이미 이펙트가 있어서 기준 소리가 달랐기 때문입니다).

JavaScript 295 — 레벨 미터 (`audio/meter.js`, `sequencer-devices.js`)

항목	내용
파일	`studionx/frontend/static/js/audio/meter.js`, `studionx/frontend/static/js/pages/sequencer-devices.js` (미터 부분)
작업	S1-11a 재생 오디오 그래프: 흉내 내던 `Math.random()` 미터를 진짜 소리 크기로 교체 (FR-251)
작성	Claude (AI), 2026-10-19, 사용자 검토
이전, 다음	이전 294 이펙트 체인 관리, 다음 296 화면 연결

1. 하는 일

트랙의 소리 크기(피크)를 좌우로 읽어서 화면 막대의 높이(0에서 100%)로 바꿉니다. 이전에는 재생 중이면 막대를 무작위로 흔들기만 했습니다.

```
트랙의 [pan] 출력 ─▶ ChannelSplitter ─▶ AnalyserNode(왼쪽)  ─▶ 피크 ─▶ dB ─▶ %
                                      └─▶ AnalyserNode(오른쪽) ─▶ 피크 ─▶ dB ─▶ %

화면 타이머 (140ms)  각 막대마다 값을 읽고, 올라갈 때는 바로, 내려갈 때는 9%씩 천천히
```

2. 언제 쓰나

쓰는 곳	하는 일
`MixerGraph.meterFor` (288)	트랙마다 `LevelTap`을 하나 만들어 볼륨과 팬을 거친 소리를 읽습니다. 마스터는 최종 출력을 읽습니다.
`PlaybackSession.meterLevels` (290)	트랙 번호로 `[왼쪽%, 오른쪽%]`를 돌려줍니다.
`sequencer-devices.js`의 `tickMeters`	140ms마다 믹서 줄과 펼침 칸의 막대 `--m` 값을 갱신합니다.
`sequencer-audio.js` (296)	`setMeterSource`로 세션의 `meterLevels`를 화면에 연결하고, 재생 시작 때 `startMeters`로 타이머를 켭니다.

3. 코드

3.1 순수 함수 (`meter.js`)

```
export function peakOf(samples) {
  let peak = 0;
  for (let i = 0; i < samples.length; i++) {
    const v = Math.abs(samples[i]);
    if (v > peak) peak = v;
  }
  return peak;
}
```

```
export function peakToPercent(peak) {
  if (!(peak > 0)) return 0;
  const db = 20 * Math.log10(peak);
  return Math.max(0, Math.min(100, (db - METER_FLOOR_DB) / -METER_FLOOR_DB * 100));
}
```

```
export function meterStep(prev, now, fall) {
  return now >= prev ? now : Math.max(now, prev - fall);
}
```

3.2 소리 크기 읽기 (`meter.js`)

```
read() {
  this.left.getFloatTimeDomainData(this.bufL);
  this.right.getFloatTimeDomainData(this.bufR);
  return [peakToPercent(peakOf(this.bufL)), peakToPercent(peakOf(this.bufR))];
}
```

3.3 화면 갱신 (`sequencer-devices.js`)

```
export function setMeterSource(fn) {
  meterSource = typeof fn === 'function' ? fn : null;
}
```

```
export function startMeters() {
  if (meterTimer) return;
  if (typeof window === 'undefined' || !window.setInterval) return;
  meterTimer = window.setInterval(tickMeters, 140);
}
```

```
function tickMeters() {
  const bars = document.querySelectorAll('.sq-acc.is-live .sq-meter i, .sq-strip.is-live .sq-meter i');
  if (!bars.length) {
    window.clearInterval(meterTimer);
    meterTimer = 0;
    return;
  }
  const playing = isPlaying();
  const cache = {};   // 이번 틱에 같은 트랙을 두 번 읽지 않게
  let busy = false;   // 막대가 하나라도 0 보다 큼
  Array.prototype.forEach.call(bars, function (bar) {
    const prev = parseFloat(bar.style.getPropertyValue('--m')) || 0;
    let now = 0;
    const owner = bar.closest('[data-track-id]');
    if (playing && owner && meterSource) {
      const id = owner.dataset.trackId;
      if (!(id in cache)) cache[id] = meterSource(id);
      const lv = cache[id];
      if (lv) now = lv[bar.matches('i:first-child') ? 0 : 1];
    }
    const next = now >= prev ? now : Math.max(now, prev - 9);   // 올라갈 때는 바로, 내려갈 때는 천천히
    if (next > 0) busy = true;
    bar.style.setProperty('--m', Math.round(next) + '%');
```

```
  });
  // 멈춰 있고 막대가 모두 0 이면 타이머를 끔 (다시 재생하면 startMeters 가 켬)
  if (!playing && !busy) {
    window.clearInterval(meterTimer);
    meterTimer = 0;
  }
}
```

4. 최신 문법으로 쓰면

```
// peakOf 와 peakToPercent 를 최신 문법으로 쓴 모양
export const peakOf = (samples) => samples.reduce((peak, v) => Math.max(peak, Math.abs(v)), 0);

export const peakToPercent = (peak) =>
  peak > 0
    ? Math.min(100, Math.max(0, ((20 * Math.log10(peak) - METER_FLOOR_DB) / -METER_FLOOR_DB) * 100))
    : 0;
```

전통 문법 (실제 코드)	최신 문법
`for` 문으로 최댓값 찾기	`reduce`
`if (!(peak > 0)) return 0;`	삼항 연산자
`function (bar) { ... }`	`(bar) => ...`

5. 왜 이렇게

선택	이유
볼륨과 팬 뒤의 소리를 읽습니다	사용자가 듣는 소리와 같은 크기를 보여야 합니다. 뮤트한 트랙은 막대가 0이 됩니다.
dB 눈금으로 바꿉니다	사람은 소리 크기를 로그로 듣기 때문에, 직선 눈금(0에서 1)으로 그리면 작은 소리가 거의 안 보입니다. -60dB를 0%, 0dB를 100%로 둡니다.
피크를 읽습니다 (RMS가 아님)	클리핑에 가까운지 보는 것이 미터의 주된 쓰임이고, 계산도 단순합니다.
올라갈 때는 바로, 내려갈 때는 천천히	순간적인 큰 소리를 놓치지 않으면서 막대가 덜덜 떨리지 않게 합니다. 한 번에 9%씩 내려갑니다.
같은 틱에 같은 트랙을 한 번만 읽습니다	아코디언과 믹서 줄이 같은 트랙을 함께 보일 때 읽기를 반복하지 않으려는 것입니다 (`cache`).
멈춰 있고 막대가 모두 0이면 타이머를 끕니다	쉬는 동안 140ms마다 도는 일을 없앱니다. 재생을 시작하면 `startMeters`가 다시 켭니다.
막대를 `data-track-id`가 있는 가장 가까운 요소에서 찾습니다	믹서 줄과 펼침 칸 모두 같은 방식으로 트랙을 알아냅니다. 화면 구조가 바뀌어도 이 규칙만 지키면 됩니다.
소리 쪽과 화면 쪽을 `setMeterSource`로 이었습니다	화면 파일이 오디오 세션을 직접 가져오면 서로 가져오는 순환이 생깁니다. 함수 하나만 주고받습니다.

6. 시험

- `docs/handoff/s1-11a_tests/meter_unit.mjs` (14건)는 순수 함수를 확인합니다. 피크는 절댓값의 최댓값이고, 1은 100%, 0은 0%이며, 0.5는 약 90%(-6dB), 0.1은 약 66.7%(-20dB)입니다. 잘못된 값(NaN, 음수)은 0%입니다. 막대는 올라갈 때 바로 오르고 내려갈 때 9%씩 내려갑니다.

- `docs/handoff/s1-11a_tests/ui_meter.mjs` (10건)는 실제 화면에서 확인합니다. 진폭 0.4(-8dB)의 사인파를 재생하면 좌우 막대가 80에서 93% 사이(약 87%)이고, 흉내처럼 흔들리지 않고 변동이 5% 이내입니다. 마스터 막대도 50%를 넘습니다. 정지하면 천천히 내려오고 2초 뒤에 0이 됩니다.
- 시험 중 발견한 일: 처음에는 막대가 계속 0이었습니다. `startMeters`가 펼침 칸을 열 때만 불려서, 믹서 줄의 타이머가 시작된 적이 없었기 때문입니다. 재생을 시작할 때 타이머를 켜고, 모두 0이면 끄도록 고쳤습니다. 이전의 흉내 미터도 같은 이유로 믹서 줄에서는 움직이지 않았습니다.

JavaScript 296 — 화면과 오디오 이어 주기 (`sequencer-audio.js` 외)

항목	내용
파일	`studionx/frontend/static/js/pages/sequencer-audio.js` (신규), `sequencer-arrange.js`, `sequencer-devices.js`, `sequencer.js` (연결 부분만 수정)
작업	S1-11a 재생 오디오 그래프: 화면의 값을 읽어 오디오 세션에 넘기고, 재생 버튼을 소리에 연결 (FR-201, FR-205)
작성	Claude (AI), 2026-10-19, 사용자 검토
이전, 다음	이전 295 레벨 미터, 다음 문서 없음 (작업 개요 82)

1. 하는 일

오디오 폴더(`audio/`)는 화면을 모르는 순수한 소리 코드입니다. 이 파일이 둘 사이를 잇습니다. 화면의 클립, 트랙, BPM, 루프를 읽어서 세션에 넘기고, ▶ ⏸ ■ 버튼이 실제 소리를 내게 합니다.

```
화면 (DOM, 서버가 준 데이터)                          소리 (audio/)
  .sq-clip--audio ┐
  .sq-clip--midi  ┤  sequencer-audio.js          ┌─▶ PlaybackSession
  트랙, 악기, 이펙트 ┼─ collect... / playback... ──▶ │     ├─ 믹서
  #bpm, Loop      ┘   (읽기만 함)                 │     ├─ 클립, 노트 재생
                                                  │     └─ 이펙트 체인
  ▶ ⏸ ■ 버튼 ─▶ setTransportHooks (sequencer-arrange.js) ─▶ onPlay / onPause
  미터 막대 ◀─ setMeterSource (sequencer-devices.js) ◀── session.meterLevels
```

2. 언제 쓰나

쓰는 곳	하는 일
`sequencer.js`	모듈을 불러온 뒤 `initAudio()`를 한 번 부릅니다.
재생 버튼	편곡의 `play()`가 `onPlay(bar)`를 부르면 세션이 그 마디부터 소리를 시작합니다.
일시정지, 정지	편곡의 `pause()`가 `onPause()`를 부르면 소리를 끊습니다.
헤드리스 시험	`audioSession()`으로 세션을 얻어 마스터 출력의 피크를 잽니다.

3. 코드

3.1 화면에서 클립 모으기

```
function collectAudioClips() {
  const list = [];
  Array.prototype.forEach.call(document.querySelectorAll('.sq-clip--audio'), function (el) {
    const trackRow = el.closest('.sq-track');
    const trackId = trackRow ? Number(trackRow.dataset.trackId) : NaN;
    const sampleId = el.dataset.sampleId ? Number(el.dataset.sampleId) : NaN;
    if (!isFinite(trackId) || !isFinite(sampleId)) return;
```

```
    const rec = el.dataset.clipId ? clipRecord(el.dataset.clipId) : null;
    const startBar = parseFloat(el.style.getPropertyValue('--s')) || 0;
    const lengthBar = parseFloat(el.style.getPropertyValue('--l')) || 0;
    const offsetBar = parseFloat(el.style.getPropertyValue('--o')) || 0;
    if (lengthBar <= 0) return;
    list.push({
      clip_id: el.dataset.clipId ? Number(el.dataset.clipId) : null,
      track_id: trackId,
      sample_id: sampleId,
      start_tick: Math.round(startBar * TICKS_PER_BAR),
      length_tick: Math.round(lengthBar * TICKS_PER_BAR),
      offset_tick: Math.round(offsetBar * TICKS_PER_BAR),
      gain_db: rec ? Number(rec.gain_db) || 0 : 0,
      is_looped: rec ? !!rec.is_looped : false,
      pitch_semitones: rec ? Number(rec.pitch_semitones) || 0 : 0,
    });
  });
  return list;
}
```

```
function collectMidiClips() {
  const list = [];
  Array.prototype.forEach.call(document.querySelectorAll('.sq-clip--midi'), function (el) {
    const trackRow = el.closest('.sq-track');
    const trackId = trackRow ? Number(trackRow.dataset.trackId) : NaN;
    if (!isFinite(trackId)) return;
    const startBar = parseFloat(el.style.getPropertyValue('--s')) || 0;
    const lengthBar = parseFloat(el.style.getPropertyValue('--l')) || 0;
    const offsetBar = parseFloat(el.style.getPropertyValue('--o')) || 0;
    if (lengthBar <= 0) return;
    const notes = el.__notes ? el.__notes : (el.dataset.patternId ? patternNotes(el.dataset.patternId) :
[]);
    list.push({
      track_id: trackId,
      start_tick: Math.round(startBar * TICKS_PER_BAR),
      length_tick: Math.round(lengthBar * TICKS_PER_BAR),
      offset_tick: Math.round(offsetBar * TICKS_PER_BAR),
      notes: notes,
    });
  });
  return list;
}
```

3.2 세션 만들기와 연결

```
function getSession() {
  if (session) return session;
  session = new PlaybackSession({
    getTracks: playbackTracks,
    getClips: collectAudioClips,
    getMidiClips: collectMidiClips,
    getInstruments: playbackInstruments,
    getEffects: playbackEffects,
    getBpm: currentBpm,
    getLoop: function () {
```

```
      const loop = getLoop();
      if (!loop.on || !(loop.length > 0)) return null;
      return { start: Math.round(loop.start * TICKS_PER_BAR), end: Math.round((loop.start + loop.length)
* TICKS_PER_BAR) };
    },
    loadSampleBytes: loadSampleBytes,
  });
  return session;
}
```

```
export function initAudio() {
  setTransportHooks({
    onPlay: function (bar) {
      const s = getSession();
      startMeters();
      s.start(Math.round(bar * TICKS_PER_BAR)).then(startRefresh).catch(function (err) {
        if (window.console) console.warn('[audio] 재생을 시작하지 못했습니다', err);
      });
    },
    onPause: function () {
      stopRefresh();
      if (session) session.pause();
    },
  });
  // 미터 막대가 진짜 소리 크기를 읽게 함 (세션이 아직 없으면 null → 막대는 0 으로 내려감)
  setMeterSource(function (trackId) { return session ? session.meterLevels(trackId) : null; });
  if (window.console) console.debug('[audio] connected, PPQ', PPQ);
}
```

3.3 편곡 쪽 연결 고리 (`sequencer-arrange.js`)

```
export function setTransportHooks(hooks) {
  transportHooks.onPlay = hooks && hooks.onPlay ? hooks.onPlay : null;
  transportHooks.onPause = hooks && hooks.onPause ? hooks.onPause : null;
}
```

3.4 서버 데이터를 재생용으로 바꾸는 읽기 함수 (`sequencer-devices.js`)

```
export function playbackTracks() {
  return tracks.map(function (t) {
    const row = document.querySelector('.sq-track[data-track-id="' + t.track_id + '"]');
    const mute = row ? row.querySelector('.is-mute') : null;
    const solo = row ? row.querySelector('.is-solo') : null;
    return Object.assign({}, t, {
      is_muted: mute ? mute.getAttribute('aria-pressed') === 'true' : !!t.is_muted,
      is_solo: solo ? solo.getAttribute('aria-pressed') === 'true' : !!t.is_solo,
    });
  });
}
```

```
export function playbackInstruments(trackId) {
  const d = devices[trackId];
  if (!d) return [];
  return (d.instruments || []).map(function (inst) {
```

```
    const plugin = pluginById(inst.plugin_id);
    return {
      id: inst.track_instrument_id,
      plugin_code: plugin ? plugin.code : '',
      schema: plugin && plugin.param_schema ? plugin.param_schema : [],
      params: inst.params || null,
      volume_db: Number(inst.volume_db) || 0,
      pan: Number(inst.pan) || 0,
      is_enabled: inst.is_enabled !== false,
    };
  });
}
```

```
export function playbackEffects(trackId) {
  const d = devices[trackId];
  if (!d) return [];
  return (d.effects || []).map(function (fx) {
    const plugin = pluginById(fx.plugin_id);
    return {
      id: fx.track_plugin_id,
      plugin_code: plugin ? plugin.code : '',
      schema: plugin && plugin.param_schema ? plugin.param_schema : [],
      params: fx.params || {},
      is_bypassed: !!fx.is_bypassed,
      slot_index: fx.slot_index,
    };
  });
}
```

```
export function patternNotes(patternId) {
  const p = patternsById[patternId];
  return p && p.notes ? p.notes : [];
}
```

```
export function clipRecord(clipId) {
  const found = clips.filter(function (c) { return String(c.clip_id) === String(clipId); })[0];
  return found || null;
}
```

4. 최신 문법으로 쓰면

```
// playbackInstruments 를 최신 문법으로 쓴 모양
export const playbackInstruments = (trackId) =>
  (devices[trackId]?.instruments ?? []).map((inst) => {
    const plugin = pluginById(inst.plugin_id);
    return {
      id: inst.track_instrument_id,
      plugin_code: plugin?.code ?? '',
      schema: plugin?.param_schema ?? [],
      params: inst.params ?? null,
      volume_db: Number(inst.volume_db) || 0,
      pan: Number(inst.pan) || 0,
      is_enabled: inst.is_enabled !== false,
```

```
    };
  });
```

전통 문법 (실제 코드)	최신 문법
`if (!d) return []; d.instruments || []`	`devices[trackId]?.instruments ?? []`
`plugin ? plugin.code : ''`	`plugin?.code ?? ''`
`function (inst) { ... }`	`(inst) => { ... }`
`Array.prototype.forEach.call(NodeList, ...)`	`[...NodeList].forEach` 또는 `for...of`

5. 왜 이렇게

선택	이유
화면을 읽는 코드를 한 파일에 모았습니다	`audio/` 는 화면(DOM)을 전혀 몰라야 Node에서 시험하고 WAV 출력에서도 쓸 수 있습니다. 화면과 닿는 부분을 이 파일 하나로 제한했습니다.
클립 위치와 길이는 화면 값을, 게인과 루프와 음정은 서버 기록을 읽습니다	사용자가 끌어서 옮긴 값은 화면이 가장 최신입니다. 화면에 없는 속성은 서버 기록에 있습니다.
재생 시작 때 스냅샷을 만듭니다	재생 중에 편집해도 이미 예약한 소리와 어긋나지 않습니다.
저장되지 않은 트랙과 샘플이 없는 클립은 건너뜁니다	서버 번호(숫자)가 없는 임시 요소는 소리를 낼 재료가 없습니다.
뮤트와 솔로는 화면 버튼 상태를 따릅니다 (`playbackTracks`)	버튼을 누르면 화면이 먼저 바뀌고 저장은 뒤따릅니다. 소리가 저장을 기다리면 반응이 늦게 느껴집니다.
피아노롤에서 고친 노트는 `__notes`를, 아니면 서버 패턴의 노트를 씁니다	피아노롤을 연 클립은 화면이 더 최신이고, 열지 않은 클립은 서버가 준 값이 맞습니다.
재생 연결을 훅(`setTransportHooks`)으로 합니다	편곡 파일이 오디오 파일을 직접 가져오면 서로 가져오는 순환이 생깁니다. 편곡은 "재생이 시작됨"만 알립니다.
샘플 내려받기에 실패해도 재생선은 움직입니다	소리 시작은 `Promise`로 처리하고 실패하면 경고만 남깁니다. 화면 재생은 소리와 독립적으로 동작합니다.
믹서 갱신 타이머를 재생 중에만 돌립니다	150ms마다 볼륨, 뮤트, 이펙트를 다시 읽습니다. 멈춰 있을 때는 돌릴 필요가 없습니다.

6. 시험

- `docs/handoff/s1-11a_tests/ui.mjs`, `ui_midi.mjs`, `ui_fx.mjs`, `ui_meter.mjs`가 모두 이 연결을 통해 실제 화면에서 동작합니다. 사인파 클립을 만들고 ▶를 눌러 마스터 출력의 피크를 읽는 방식입니다.
- 시험 설계에서 배운 점: 예시 화면의 재생선이 처음부터 0마디가 아니고, 기존 트랙에 이미 이펙트가 있고 볼륨이 낮을 수 있습니다. 그래서 시험은 늘 전용 트랙과 새 Synth를 만들고, 시작 전에 ■로 재생선을 0으로 되돌리며, 끝나면 모두 지웁니다.